BazzBasic

Language Reference Manual

Version 1.1 · BASIC Interpreter for .NET 10

---

ekbass.github.io/BazzBasic

Kristian Virtanen · MIT License

BazzBasic

BazzBasic is a BASIC interpreter built to work with the .NET 10 Framework.

It supports many of the features of BASIC interpreters from the 80s, but also offers something modern.

Development

So far, EkBass (https://github.com/EkBass) has been responsible for the development of BazzBasic. BazzBasic is released under the open source MIT license. Its source code is available and visible to everyone in the project's GitHub repository.

Currently, the development work is done in the Windows 11 operating system, but with quite a bit of effort it can also be translated to Linux or MacOS.

Main functionalities

Most familiar BASIC features work either completely or almost completely as users of traditional BASIC languages are used to using them.

User-Defined Functions

```
DEF FN factorial$(n$)
    IF n$ <= 1 THEN
        RETURN 1
    END IF
    RETURN n$ * FN factorial$(n$ - 1)
END DEF

PRINT FN factorial$(5)   ' Output: 120
PRINT FN factorial$(10)  ' Output: 3628800
```

SDL2 Graphics

BazzBasic offers a reasonable sampling of SDL2 features. If your program uses graphic features, SDL2.dll must be in the same directory.

Sounds

BazzBasic includes a sound system built on SDL2_mixer, supporting audio playback with both background and blocking modes.

Source Control

With the INCLUDE function, you can split the source code into different files and folders or generate tokenized libraries.

Typeless Variables and Constants

```
LET num$ = 42            ' Number
LET text$ = "Hello"      ' String
LET mixed$ = "123"       ' String (quoted)
```

BazzBasic size

Currently, BazzBasic requires about 70 megabytes + SDL2.dll.

BazzBasic includes .NET 10 assemblies during compilation, which affects the file size. .NET 10, although a bit bulky, still offers compatibility far into the future.

Links

- **Homepage:** https://ekbass.github.io/BazzBasic/
- **GitHub:** https://github.com/EkBass/BazzBasic
- **Examples:** https://github.com/EkBass/BazzBasic/tree/main/Examples
- **Discussions:** https://github.com/EkBass/BazzBasic/discussions

Contact & License

Author of BazzBasic is **Kristian Virtanen**.

Email: krisu.virtanen@gmail.com
URL: https://github.com/EkBass/BazzBasic

License

BazzBasic is released under **MIT** license.

> *MIT License — Copyright (c) 2026 Kristian Virtanen*
>
> *Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:*
>
> *The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.*
>
> *THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.*

Getting Started

Installation

Download binary

- Download BazzBasic binary from GitHub Releases: https://github.com/EkBass/BazzBasic/releases
- Extract the `.zip` package to a folder of your choice.

Firewall issues

Since BazzBasic is built around .NET 10 and it is still a fairly recent project, Windows Firewall or third-party security programs may give a warning about it the first time you use it.

The truth is that open source projects that use .NET platforms will always suffer from this problem to some extent.

Run BazzBasic IDE

- Browse to folder where you extracted `bazzbasic.exe`
- Double-click `bazzbasic.exe` and IDE opens.

In order to run your BazzBasic code with the built-in IDE, you must save your file first and then press **F5** to run it.

Use external IDE

- BazzBasic syntax highlighting is available for *Notepad++*, *Geany* and *Visual Studio Code*.
- Get config files from: https://github.com/EkBass/BazzBasic/tree/main/extras
- Adjust your editor to launch `bazzbasic.exe <filename.bas>`

Use with terminal

Command	Action
`bazzbasic.exe`	Launch IDE
`bazzbasic.exe <filename.bas>`	Run program
`bazzbasic.exe -exe <filename.bas>`	Create standalone .exe
`bazzbasic.exe -lib <filename.bas>`	Create tokenized library (.bb)

IDE Usage

BazzBasic includes a simple integrated development environment (IDE).

Starting the IDE

Run `BazzBasic.exe` without any arguments to open the IDE.

Editor Features

- **Syntax highlighting** for BazzBasic keywords
- **Line numbers** for easy navigation
- **Multiple tabs** for working with several files

Keyboard Shortcuts

Shortcut	Action
F5	Run program
Ctrl+N	New file
Ctrl+O	Open file
Ctrl+S	Save file
Ctrl+Shift+S	Save As
Ctrl+W	Close tab
Ctrl+F	Find
Ctrl+H	Replace

Running Programs

Press **F5** to run the current program. A console window will open showing the output. If there are errors, they will be displayed with line numbers.

Creating Standalone Executables

BazzBasic can package your BASIC program into a standalone .exe file.

Usage

```
bazzbasic.exe -exe yourprogram.bas
```

This creates `yourprogram.exe` in the same folder as the .bas file.

Required Files for Distribution

File	Required	Purpose
yourprogram.exe	Yes	Your packaged program
SDL2.dll	Yes	Graphics and input
SDL2_mixer.dll	Yes	Audio

Distribution with Assets

```
mygame/
├── yourprogram.exe
├── SDL2.dll
├── images/
│   ├── player.png
│   └── enemy.png
└── sounds/
    ├── shoot.wav
    └── music.mp3
```

Notes

- INCLUDE files are processed at packaging time (embedded in exe)
- File paths in your BASIC code are relative to the exe location
- The packaged exe contains the full BazzBasic interpreter

Creating Libraries

BazzBasic supports pre-compiled libraries (`.bb` files) for code reuse. Libraries contain tokenized functions that load faster than source files.

What is a Library?

A library is a collection of user-defined functions compiled into a binary format.

Creating a Library

Libraries can **only contain `DEF FN` functions**.

```
REM MathLib.bas
DEF FN add$(x$, y$)
    RETURN x$ + y$
END DEF

DEF FN multiply$(x$, y$)
    RETURN x$ * y$
END DEF
```

Compile with: `bazzbasic.exe -lib MathLib.bas`

Using a Library

```
INCLUDE "MathLib.bb"

PRINT "5 + 3 = "; FN MATHLIB_add$(5, 3)
PRINT "5 * 3 = "; FN MATHLIB_multiply$(5, 3)
```

Naming Convention

Source file	Function prefix
`MathLib.bas`	`MATHLIB_`
`StringUtils.bas`	`STRINGUTILS_`

Library Rules

1. **Functions only** — no variables, constants, or loose code
2. **Automatic prefix** — all functions get the library name as prefix
3. **Binary format** — `.bb` files are not human-readable
4. **Version locked** — libraries compiled with one version may not work with others
5. **Constant access** — library functions can read constants from the main program

Language Reference

Variables & Constants

Variables store values that can change during program execution. Constants store values that cannot be changed.

- Variables need the suffix `$`
- Constants need the suffix `#`
- Both must be initialized with `LET` before use

Variables

```
LET a$ = "Foo"
LET b$ = 1
LET c$ = 1 + 2
```

Constants

```
LET PI# = 3.14159
LET MAX_PLAYERS# = 4
LET GAME_TITLE# = "Space Invaders"
```

LET only on init

```
LET a$ = 1
a$ = 3      ' No LET needed after init
```

Typeless variables

```
LET a$ = 1
a$ = "Foo"  ' OK — variable changes type
```

Multiple declarations

```
LET a$ = 1, b$ = 2
LET c$ = a$ + b$
```

Exceptions: FOR and INPUT

```
INPUT "What is your name? ", name$  ' No LET needed
FOR i$ = 1 TO 10                    ' No LET needed
    PRINT i$
NEXT
```

Comparison

```
LET a$ = "123"
LET b$ = 123
IF a$ = b$ THEN PRINT "Same"  ' TRUE — cross-type comparison
```

Scope

Variables in the main code share one global scope. User-defined functions have completely isolated scope — only global constants (`#`) are accessible inside functions.

```
LET b$ = 100
LET C# = "Foo"

DEF FN test$(a$)
    LET b$ = 2        ' Local b$ — separate from global
    PRINT C#          ' Global constants are available
    RETURN a$ * b$
END DEF
```

Built-in Constants: Keyboard

Arrow Keys		
KEY_UP#	KEY_DOWN#	KEY_LEFT#
KEY_RIGHT#		

Special Keys		
KEY_ESC#	KEY_TAB#	KEY_BACKSPACE#
KEY_ENTER#	KEY_SPACE#	KEY_INSERT#
KEY_DELETE#	KEY_HOME#	KEY_END#

Function Keys		
KEY_F1#–KEY_F6#	KEY_F7#–KEY_F12#	

Alphabet		
KEY_A#–KEY_Z#		

Number Keys		
KEY_0#–KEY_9#		

Naming Rules

- Must end with `$` (variable) or `#` (constant)
- Can contain letters, numbers, and underscores
- Cannot start with a number
- Case-insensitive

Arrays & JSON

Arrays store collections of values. BazzBasic arrays are fully dynamic and support numeric, string, or mixed indexing. All array names must end with `$`.

Initialization

```
DIM scores$
DIM a$, b$, c$   ' Multiple declarations
```

Numeric Indexing

```
DIM scores$
scores$(0) = 95
scores$(1) = 87
PRINT scores$(0)   ' Output: 95
```

String Indexing (Associative)

```
DIM player$
player$("name") = "Alice"
player$("score") = 1500
PRINT player$("name")   ' Output: Alice
```

Multi-dimensional

```
DIM matrix$
matrix$(0, 0) = "A1"
matrix$(0, 1) = "A2"
matrix$(1, 0) = "B1"
```

Array Functions

Function	Description
`LEN(arr$())`	Element count (note empty parens)
`HASKEY(arr$(key))`	1 if exists, 0 if not
`DELKEY arr$(key)`	Remove element
`DELARRAY arr$`	Remove entire array

```
DIM items$
items$(0) = "apple"
items$(1) = "banana"
PRINT LEN(items$())      ' Output: 2

IF HASKEY(items$(0)) THEN
    PRINT "Key 0 exists"
END IF
```

Arrays and JSON

BazzBasic arrays map naturally to JSON. Nested JSON objects become comma-separated keys.

ASJSON

```
DIM player$
player$("name") = "Alice"
player$("score") = 9999
player$("address,city") = "New York"
player$("skills,0") = "JavaScript"
player$("skills,1") = "Python"

LET json$ = ASJSON(player$)
' Output: {"name":"Alice","score":9999,"address":{"city":"New York"},"skills":
["JavaScript","Python"]}
```

ASARRAY

```
DIM data$
LET count$ = ASARRAY(data$, "{""name"":""Bob"",""score"":42}")
PRINT data$("name")    ' Output: Bob
PRINT data$("score")   ' Output: 42
```

LOADJSON / SAVEJSON

```
DIM save$
save$("level") = 3
save$("hp") = 80
SAVEJSON save$, "savegame.json"

DIM loaded$
LOADJSON loaded$, "savegame.json"
```

Loading key=value Files

When `FileRead` assigns to a `DIM`'d array, BazzBasic automatically parses `key=value` lines:

```
DIM config$
LET config$ = FileRead("settings.txt")
PRINT config$("width")    ' Output: 800
```

Works with `.env` files too:

```
DIM env$
LET env$ = FileRead(".env")
LET ApiKey# = env$("OPENAI_API_KEY")
```

Operators

Arithmetic

Operator	Description
`+`	Addition or string concatenation
`-`	Subtraction
`*`	Multiplication
`/`	Division (returns float)

Comparison

Operator	Description
`=` or `==`	Equal
`<>` or `!=`	Not equal
`<`	Less than
`>`	Greater than
`<=`	Less or equal
`>=`	Greater or equal

Logical

Operator	Description
`AND`	Logical AND (not bitwise)
`OR`	Logical OR
`NOT`	Logical NOT

Important: `AND` is **logical**, not bitwise, in BazzBasic.

Operator Precedence (high to low)

1. `()` — Parentheses
2. `NOT` — Logical NOT
3. `*`, `/` — Multiplication, Division
4. `+`, `-` — Addition, Subtraction
5. `=`, `<>`, `<`, `>`, `<=`, `>=` — Comparison
6. `AND` — Logical AND
7. `OR` — Logical OR

TRUE and FALSE

```
LET gameOver$ = FALSE

WHILE NOT gameOver$
    IF lives$ = 0 THEN gameOver$ = TRUE
WEND
```

Comments

BazzBasic supports two comment styles.

REM

```
REM This is a comment
REM Initialize player position
LET x$ = 100
```

Single Quote

```
' This is a comment
LET x$ = 100    ' Inline comment
```

Both styles are completely identical in behavior. Single quote `'` is generally preferred for its brevity.

```
' ============================================
' player.bas — Player movement logic
' ============================================

[inits]
    LET x$ = 320    ' Start at center X
    LET y$ = 240    ' Start at center Y
```

User Defined Functions

Basic Syntax

The function name **must** have `$` as suffix.

```
DEF FN functionName$(param1$, param2$)
    ' Function body
    RETURN value
END DEF
```

Functions must always return a value, and the return value must be used:

```
PRINT FN double$(21)        ' OK — return value goes to PRINT
LET foo$ = FN double$(5)    ' OK — return value stored in variable
FN double$(21)              ' ERROR — return value has nowhere to go
```

Examples

```
DEF FN double$(x$)
    RETURN x$ * 2
END DEF
PRINT FN double$(21)    ' Output: 42

DEF FN add$(a$, b$)
    RETURN a$ + b$
END DEF
PRINT FN add$(3, 4)     ' Output: 7
```

Recursive Functions

```
DEF FN factorial$(n$)
    IF n$ <= 1 THEN
        RETURN 1
    END IF
    RETURN n$ * FN factorial$(n$ - 1)
END DEF
PRINT FN factorial$(5)    ' Output: 120
```

Scope

Functions have completely isolated scope:

- Parameters and local variables are invisible outside the function
- Functions **cannot** access or modify global variables
- Global constants (`#`) are available inside functions

- Parameters are passed **by value**

Labels Inside Functions

GOTO and GOSUB inside a function **cannot** jump outside the function. This restriction ensures functions are self-contained.

Order of Code

Functions must be defined **before** they are called. Put functions at the top of the file, or use INCLUDE.

Passing Array Values

Arrays cannot be passed directly. Pass individual element values:

```
DIM a$
a$("name") = "Foo"
a$("age") = 19

DEF FN func$(name$, age$)
    PRINT name$
    PRINT age$
    RETURN 0
END DEF

LET b$ = FN func$(a$("name"), a$("age"))
```

Control Flow

IF Statements

```
IF score$ >= 90 THEN
    PRINT "A"
ELSEIF score$ >= 80 THEN
    PRINT "B"
ELSE
    PRINT "C"
END IF
```

One-line IF (GOTO/GOSUB only):

```
IF lives$ = 0 THEN GOTO [game_over]
IF key$ = KEY_ESC# THEN GOTO [menu] ELSE GOTO [continue]
```

FOR Loops

FOR auto-declares the loop variable — no LET needed.

```
FOR i$ = 1 TO 10
    PRINT i$
NEXT

FOR i$ = 0 TO 100 STEP 10
    PRINT i$
NEXT

FOR i$ = 10 TO 1 STEP -1     ' Count down
    PRINT i$
NEXT
```

WHILE Loops

```
LET count$ = 0
WHILE count$ < 5
    PRINT count$
    count$ = count$ + 1
WEND
```

Labels and GOTO

```
[start]
    PRINT "Starting..."
    GOTO [main]

[main]
    PRINT "Main section"
    END
```

GOSUB and RETURN

```
GOSUB [greet]
END

[greet]
    PRINT "Hello!"
    RETURN
```

Dynamic Jumps

```
LET target$ = "[menu]"
GOTO target$              ' Variable must contain "[label]" format

LET dest# = "[jump]"
GOSUB dest#
```

END and SLEEP

```
IF error$ THEN
    PRINT "Error!"
    END
END IF

SLEEP 2000   ' Pause 2 seconds
```

Statements A–Z

CLS

Clear the screen.

```
CLS
```

COLOR

Set text foreground and background colors (0–15 palette).

```
COLOR 14, 1    ' Yellow text on blue background
PRINT "Colorful!"
```

INPUT and LINE INPUT

```
INPUT "Enter name: ", firstName$, lastName$
LINE INPUT "Enter full line: ", name$
```

Feature	INPUT	LINE INPUT
Reads spaces	No (splits)	Yes
Multiple variables	Yes	No
Default prompt	`"? "`	None

LOCATE

Move cursor to specified position (1-based).

```
LOCATE 10, 20
PRINT "Positioned text"
```

PRINT

```
PRINT "Hello, World!"
PRINT "Value: "; x$
PRINT "A", "B", "C"    ' Tab-separated
PRINT "X="; 10;        ' No newline (trailing semicolon)
```

Escape characters: `\"` `\n` `\t` `\\`

SHELL

```
LET a$ = SHELL("dir *.txt")
LET a$ = SHELL("dir *.txt", 2000)   ' 2 second timeout
```

SLEEP

```
SLEEP 2000   ' Pause 2000 milliseconds
```

Functions

Math Functions

Function	Description
`ABS(n)`	Absolute value
`ATAN(n)`	Arc tangent
`BETWEEN(n, min, max)`	TRUE if n in range
`CEIL(n)`	Round up
`CINT(n)`	Convert to integer
`COS(n)`	Cosine
`CLAMP(n, min, max)`	Constrain to range
`DEG(radians)`	Radians to degrees
`DISTANCE(x1,y1,x2,y2)`	2D Euclidean distance
`DISTANCE(x1,y1,z1,x2,y2,z2)`	3D Euclidean distance
`EXP(n)`	e^n
`FLOOR(n)`	Round down
`INT(n)`	Truncate toward zero
`LERP(start, end, t)`	Linear interpolation
`LOG(n)`	Natural logarithm
`MAX(a, b)`	Higher of two
`MIN(a, b)`	Lower of two
`MOD(a, b)`	Remainder
`POW(base, exp)`	Power
`RAD(degrees)`	Degrees to radians
`RND(n)`	Random 0 to n-1
`ROUND(n)`	Standard rounding
`SGN(n)`	Sign (-1, 0, 1)
`SIN(n)`	Sine
`SQR(n)`	Square root
`TAN(n)`	Tangent

Math Constants

Constant	Value	Degrees
`QPI`	0.785398...	45°
`HPI`	1.570796...	90°
`PI`	3.141592...	180°
`TAU`	6.283185...	360°
`EULER`	2.718281...	e

LERP Example

```
PRINT LERP(0, 100, 0.5)    ' Output: 50

' Smooth camera
cameraX$ = LERP(cameraX$, targetX$, 0.1)
```

DISTANCE Example

```
LET dist$ = DISTANCE(playerX$, playerY$, enemyX$, enemyY$)
IF dist$ < 50 THEN PRINT "Enemy is close!"
```

String Functions

Function	Description
`ASC(s$)`	Character to ASCII code
`CHR(n)`	ASCII code to character
`INSTR(s$, search$)`	Find substring (0 = not found)
`INSTR(start, s$, search$)`	Find from position
`INVERT(s$)`	Reverse string
`LCASE(s$)`	Lowercase
`LEFT(s$, n)`	First n characters
`LEN(s$)`	String length
`LTRIM(s$)`	Trim left
`MID(s$, start)`	Substring from position
`MID(s$, start, len)`	Substring with length
`REPEAT(s$, n)`	Repeat string n times
`REPLACE(s$, old$, new$)`	Replace all occurrences
`RIGHT(s$, n)`	Last n characters
`RTRIM(s$)`	Trim right
`SHA256(s$)`	SHA-256 hash (hex)
`SPLIT(arr$, s$, delim$)`	Split into array
`SRAND(n)`	Random string of length n
`STR(n)`	Number to string
`TRIM(s$)`	Trim both ends
`UCASE(s$)`	Uppercase
`VAL(s$)`	String to number
`BASE64ENCODE(s$)`	Encode to Base64
`BASE64DECODE(s$)`	Decode from Base64

SPLIT Example

```
DIM parts$
LET count$ = SPLIT(parts$, "apple,banana,orange", ",")
```

```
PRINT parts$(0)   ' apple
PRINT parts$(1)   ' banana
PRINT parts$(2)   ' orange
```

Input Functions

INKEY

Non-blocking key check. Returns 0 if no key pressed.

```
[loop]
    LET k$ = INKEY
    IF k$ = 0 THEN GOTO [loop]
    IF k$ = KEY_ESC# THEN END
    PRINT "Key: "; k$
    GOTO [loop]
```

KEYDOWN

Returns TRUE while key is held down. Only available in graphics mode.

```
SCREEN 12
WHILE INKEY <> KEY_ESC#
    IF KEYDOWN(KEY_LEFT#)  THEN x$ = x$ - 2
    IF KEYDOWN(KEY_RIGHT#) THEN x$ = x$ + 2
    IF KEYDOWN(KEY_UP#)    THEN y$ = y$ - 2
    IF KEYDOWN(KEY_DOWN#)  THEN y$ = y$ + 2
WEND
```

WAITKEY

Halts execution until one of the specified keys is pressed.

```
PRINT "Press ENTER to continue"
WAITKEY(KEY_ENTER#)

LET k$ = WAITKEY(KEY_A#, KEY_B#, KEY_ESC#)
PRINT "You pressed: "; k$

LET k$ = WAITKEY()   ' Any key
```

Feature	INKEY	KEYDOWN	WAITKEY
Blocking	No	No	Yes
Returns	Key value or 0	TRUE/FALSE	Key value
Use case	Game loops	Held keys	Menus

Mouse (Graphics Mode Only)

Function	Description
`MOUSEX` / `MOUSEY`	Cursor position
`MOUSELEFT`	1 if left button pressed
`MOUSERIGHT`	1 if right button pressed
`MOUSEMIDDLE`	1 if middle button pressed

```
IF MOUSELEFT   THEN PRINT "Left clicked"
IF MOUSERIGHT  THEN PRINT "Right clicked"
```

Fast Trigonometry

For graphics-intensive applications, BazzBasic provides fast trigonometric functions using pre-calculated lookup tables. ~20x faster than `SIN` / `COS` , with 1-degree precision.

Memory: ~5.6 KB when enabled.

FastTrig(enable)

Must call `FastTrig(TRUE)` before using FastSin, FastCos, or FastRad.

```
FastTrig(TRUE)

LET x$ = FastCos(45)
LET y$ = FastSin(45)

FastTrig(FALSE)   ' Free memory when done
```

FastSin / FastCos

Angle in degrees, automatically normalized to 0–359.

```
FastTrig(TRUE)

PRINT FastSin(90)    ' Output: 1
PRINT FastCos(0)     ' Output: 1
PRINT FastSin(450)   ' Same as FastSin(90) = 1

FastTrig(FALSE)
```

FastRad

Convert degrees to radians. Does **not** require `FastTrig(TRUE)` .

```
PRINT FastRad(90)    ' Output: 1.5707... (HPI)
PRINT FastRad(180)   ' Output: 3.1415... (PI)
```

When to Use

Use FastTrig when: raycasting, rotating sprites, particle systems, any loop calling trig hundreds of times per frame.

Use regular SIN/COS when: high precision needed, one-time calculations.

Other Functions

RGB(r, g, b)

Creates a color value from red, green, blue components (0–255 each).

```
LET red$ = RGB(255, 0, 0)
LET orange$ = RGB(255, 165, 0)
PSET (100, 100), orange$
```

Store colors in constants for performance:

```
LET COLOR_PLAYER# = RGB(0, 128, 255)
LET COLOR_ENEMY# = RGB(255, 64, 64)
```

TIME(format$)

Returns current date/time as a formatted string.

```
PRINT TIME()                ' "16:30:45"
PRINT TIME("dd.MM.yyyy")    ' "09.01.2026"
PRINT TIME("yyyy-MM-dd")    ' "2026-01-09"
PRINT TIME("dddd")          ' "Friday"
```

Code	Description	Example
HH	Hour (00–23)	16
mm	Minutes	30
ss	Seconds	45
dd	Day	09
MM	Month number	01
MMMM	Month name	January
yyyy	Year	2026
dddd	Weekday	Friday

TICKS

Returns milliseconds elapsed since program started.

```
LET start$ = TICKS

FOR i$ = 1 TO 10000
```

```
    LET x$ = x$ + 1
NEXT

LET elapsed$ = TICKS - start$
PRINT "Time taken: "; elapsed$; " ms"
```

GETCONSOLE(row, col, type)

Reads a character or color from a console screen position. Console only — not available in graphics mode.

- type 0 = character (ASCII code)
- type 1 = foreground color
- type 2 = background color

File Handling

BazzBasic includes a simple but powerful file system for reading and writing text files.

Functions and Commands

Function/Command	Description
`FileRead(path)`	Read file; returns string or populates array
`FileWrite path, data`	Write string or array to file
`FileAppend path, data`	Append to file
`FileExists(path)`	Returns 1 if file exists, 0 if not
`FileDelete path`	Delete a file
`FileList(path$, arr$)`	List files in directory into array

FileRead and FileWrite

```
' Simple text
FileWrite "save.txt", data$
LET data$ = FileRead("save.txt")

' Array (key=value format)
DIM a$
a$("name") = "player1"
a$("score") = 9999
FileWrite "scores.txt", a$

DIM b$
LET b$ = FileRead("scores.txt")
PRINT b$("name")   ' Output: player1
```

File Existence Check

```
IF FileExists("highscore.txt") = 1 THEN
    LET scores$ = FileRead("highscore.txt")
ELSE
    LET scores$ = "0"
END IF
```

.env Files

```
DIM env$
LET env$ = FileRead(".env")
LET ApiKey# = env$("OPENAI_API_KEY")
```

Lines starting with `#` in key=value files are treated as comments.

Path Notes

- Use forward slashes `/` or escaped backslashes `\\`
- `ROOT#` constant contains the program's base directory
- Parent directories are created automatically when writing

Best Practices

1. Check file existence before reading
2. Use relative paths for game files
3. Use `FileAppend` for logs
4. Clean up temporary files at exit

Graphics Commands

Screen Setup

```
SCREEN 12                          ' 640×480 VGA
SCREEN 0, 800, 600, "My Game"      ' Custom size + title
```

Mode	Resolution
1	320×200
9	640×350
12	640×480 (recommended)
13	320×200

Fullscreen and VSync

```
FULLSCREEN TRUE     ' Borderless fullscreen
FULLSCREEN FALSE    ' Windowed mode

VSYNC(TRUE)         ' Cap to monitor refresh (default)
VSYNC(FALSE)        ' Unlimited FPS
```

Double Buffering (Required for Animation)

```
SCREENLOCK ON
' Draw commands here
SCREENLOCK OFF
SLEEP 16       ' ~60 FPS
```

Drawing Primitives

Command	Description
`PSET (x, y), color`	Draw pixel
`POINT(x, y)`	Read pixel color
`LINE (x1,y1)-(x2,y2), color`	Draw line
`LINE (x1,y1)-(x2,y2), color, B`	Box outline
`LINE (x1,y1)-(x2,y2), color, BF`	Filled box
`CIRCLE (cx, cy), r, color`	Circle outline

`CIRCLE (cx, cy), r, color, 1`	Filled circle
`PAINT (x, y), fill, border`	Flood fill
`RGB(r, g, b)`	Create color value

Color Palette (0–15)

0 Black	4 Red	8 Dark Gray	12 Light Red
1 Blue	5 Magenta	9 Light Blue	13 Light Magenta
2 Green	6 Brown	10 Light Green	14 Yellow
3 Cyan	7 Light Gray	11 Light Cyan	15 White

Shape/Sprite System

```
' Create shape
LET sprite$ = LOADSHAPE("RECTANGLE", 50, 50, RGB(255,0,0))
' Types: RECTANGLE, CIRCLE, TRIANGLE

' Load image
LET img$ = LOADIMAGE("player.png")
LET img$ = LOADIMAGE("https://example.com/sprite.png")  ' URL

' Transform and draw
MOVESHAPE sprite$, x$, y$
ROTATESHAPE sprite$, angle$
SCALESHAPE sprite$, 1.5
SHOWSHAPE sprite$
HIDESHAPE sprite$
DRAWSHAPE sprite$
REMOVESHAPE sprite$   ' Free memory
```

Sprite Sheets

```
DIM sprites$
LOADSHEET sprites$, 128, 128, "sheet.png"

MOVESHAPE sprites$(1), x#, y#
DRAWSHAPE sprites$(1)
```

Sprites are indexed 1-based, left-to-right, top-to-bottom.

Performance Tips

1. Use `SCREENLOCK ON/OFF` for all animation
2. `LINE...BF` is faster than `CLS` for clearing

3. Store `RGB()` values in constants
4. Always `REMOVESHAPE` when done
5. Do math/logic **outside** the SCREENLOCK block

```
SCREEN 12
WHILE INKEY <> KEY_ESC#
    ' Logic here (outside SCREENLOCK)

    SCREENLOCK ON
    LINE (0,0)-(640,480), 0, BF   ' Fast clear
    CIRCLE (x$, y$), 10, RGB(0,255,0), 1
    SCREENLOCK OFF
    SLEEP 16
WEND
```

Sound System

BazzBasic includes a comprehensive sound system built on SDL2_mixer.

Sound Commands

Command	Description
`LOADSOUND(path)`	Load sound file, returns ID
`SOUNDONCE(id$)`	Play once (background)
`SOUNDONCEWAIT(id$)`	Play once (blocking)
`SOUNDREPEAT(id$)`	Loop continuously
`SOUNDSTOP(id$)`	Stop specific sound
`SOUNDSTOPALL`	Stop all sounds

Supported formats: WAV (recommended), MP3, others via SDL2_mixer.

Example

```
SCREEN 12
LET bgm$ = LOADSOUND("music.wav")
LET sfx$ = LOADSOUND("jump.wav")

SOUNDREPEAT(bgm$)   ' Start background music

WHILE INKEY <> KEY_ESC#
    IF INKEY = KEY_SPACE# THEN SOUNDONCE(sfx$)
    SLEEP 16
WEND

SOUNDSTOPALL
END
```

Best Practices

1. Load sounds once at program startup
2. Use `SOUNDONCE` for short effects
3. Use `SOUNDREPEAT` for background music
4. Use `SOUNDONCEWAIT` for narration / sequential audio
5. Always call `SOUNDSTOPALL` before exit

Network (HTTP)

HTTPGET

```
LET response$ = HTTPGET("https://api.example.com/data")

' With custom headers
DIM headers$
headers$("Authorization") = "Bearer mytoken"
LET response$ = HTTPGET("https://api.example.com/data", headers$)
```

HTTPPOST

```
LET result$ = HTTPPOST("https://api.example.com/post", "{""key"":""value""}")

' With headers
DIM headers$
headers$("Authorization") = "Bearer mytoken"
headers$("Content-Type") = "application/json"
LET result$ = HTTPPOST("https://api.example.com/post", body$, headers$)
```

HTTP + JSON API Pattern

```
DIM env$
LET env$ = FileRead(".env")
LET API_KEY# = env$("OPENAI_API_KEY")

DIM headers$
headers$("Authorization") = "Bearer " + API_KEY#
headers$("Content-Type") = "application/json"

DIM body$
body$("model") = "gpt-4.1-mini"
body$("messages,0,role") = "user"
body$("messages,0,content") = "Hello"
body$("max_tokens") = 100

LET raw$ = HTTPPOST("https://api.openai.com/v1/chat/completions", ASJSON(body$), headers$)

DIM result$
ASARRAY result$, raw$
PRINT result$("choices,0,message,content")
```

Encoding & Hashing

Function	Description
`BASE64ENCODE(s$)`	Encode to Base64
`BASE64DECODE(s$)`	Decode from Base64
`SHA256(s$)`	SHA-256 hex hash

Preprocessor & Source Control

INCLUDE

Include another BazzBasic source file. Contents are inserted at the exact point of the INCLUDE statement.

```
INCLUDE "utils.bas"
INCLUDE "graphics_helpers.bas"
INCLUDE "MathLib.bb"    ' Compiled library
```

Example

main.bas:

```
INCLUDE "helpers.bas"
LET result$ = FN Square$(5)
PRINT result$    ' Output: 25
```

helpers.bas:

```
DEF FN Square$(n$)
    RETURN n$ * n$
END DEF
```

Compiled Libraries (.bb)

```
INCLUDE "MathLib.bb"
PRINT FN MATHLIB_Square$(5)    ' Functions prefixed with FILENAME_
```

- Libraries can only contain `DEF FN` functions
- Functions auto-prefixed with the filename: `MATHLIB_functionname$`
- Library functions can read global constants (`#`) from the main program
- `.bb` files are version-locked

Recommended Program Structure

```
' main.bas — recommended layout

' 1. INCLUDE constants and functions
INCLUDE "constants.bas"
INCLUDE "functions.bas"

' 2. Init
[inits]
    SCREEN 0, 640, 480, "My Game"
    LET x$ = 100
```

```
    LET y$ = 100
    LET running$ = TRUE

' 3. Main loop
[main]
    WHILE running$
        IF INKEY = KEY_ESC# THEN running$ = FALSE
        GOSUB [sub:update]
        GOSUB [sub:draw]
        SLEEP 16
    WEND
END

' 4. Subroutines
[sub:update]
    IF KEYDOWN(KEY_LEFT#)  THEN x$ = x$ - 5
    IF KEYDOWN(KEY_RIGHT#) THEN x$ = x$ + 5
RETURN

[sub:draw]
    SCREENLOCK ON
    LINE (0,0)-(640,480), 0, BF
    CIRCLE (x$, y$), 10, RGB(0,255,0), 1
    SCREENLOCK OFF
RETURN
```